

Lezione — FI2 02: Linguaggio e piattaforma

Corso: Fondamenti di Informatica T-2 (12 CFU · S1) **Materiale:** [materiali/slide/02-x1-Linguaggi e piattaforme.pdf](#) — E. Denti, *OOP: linguaggi e piattaforme*, a.a. 2023/24, 74 slide **Prerequisiti:** 01 ✓ (eseguibile relativo all'infrastruttura, JDK/JRE, collegamento dinamico). Presuppone il C: `main`, `argc/argv`, `printf`, `#include <math.h>`, tipi `int/float/char`.

Annata: slide 2023/24, l'edizione d'iscrizione (`fonti.md`). La nota su .NET 8 (sl. 31) è una fotografia di quell'anno. La forma del `main` presentata qui è quella classica: le novità di Java 21 stanno nell'addendum 02z, modulo separato, non letto per questa lezione.

Quattro linguaggi, un esame. Le slide confrontano sistematicamente Java con C#, Scala e Kotlin. All'esame si scrive **solo Java** (JDK, Eclipse, JUnit: `fonti.md`). Il confronto non va memorizzato riga per riga: serve a far vedere *perché* Java fa le scelte che fa. Dove la lezione nomina gli altri linguaggi, lo fa per questo.

Cosa non è coperto dal testo estratto: alcune slide sono solo schermate — gli ambienti online (sl. 37), gli esperimenti in C#, Scala e Kotlin (sl. 57-59), le tabelle Unicode (sl. 62-64), gli editor (sl. 73-74). Dove servono, la lezione descrive solo ciò che si legge nell'immagine.

Obiettivo

Saper spiegare perché in Java si scrive `comp.operation(argomenti)` e non `operation(comp, argomenti)`, saper scrivere, compilare ed eseguire da riga di comando la più semplice applicazione Java sapendo cosa produce ogni passo, e saper dire quando un assegnamento fra tipi numerici è lecito, quando richiede un cast e perché — fino a cosa è davvero un `char`.

1. L'inversione del punto di vista: da `operation(comp, ...)` a `comp.operation(...)`

Cosa produce. Una notazione. In C, per svolgere un'operazione su un componente, si scrive `operation(comp, argomenti)`: «COSA fare», «CHI la fa», «dati necessari», in quest'ordine; l'esempio del docente è `fprintf(fout, "Hello!");` [fonte: 02, sl. 2]. Nei linguaggi a oggetti lo stesso gesto diventa `comp.operation(argomenti)`: «a CHI mi rivolgo», «COSA gli chiedo di fare», «dati necessari per farla» [fonte: 02, sl. 5].

Cosa c'è dietro. Non è una scelta estetica, è un cambio di enfasi. Nell'approccio procedurale «l'enfasi è sull'operazione da svolgere (primo argomento)» e «“chi” la svolge è in secondo piano (se c'è...)» [fonte: 02, sl. 2]. Questo è «naturale in un mondo semplice, dove c'è un solo (“ovvio”) destinatario delle operazioni»: un'architettura monolitica, con il focus sull'algoritmo. Ma «mostra tutti i suoi limiti in presenza di sistemi software»: molte entità interagiscono, il focus non è più solo sulle operazioni

ma «su CHI faccia COSA», su «come distribuire le responsabilità fra i componenti», e quindi nasce la «necessità di evidenziare A CHI CI SI RIVOLGE per richiedere una certa operazione / un certo SERVIZIO» [fonte: 02, sl. 3].

Nell'approccio a oggetti l'enfasi passa sull'**oggetto**: «una entità dotata di una propria identità, con le sue proprietà, e in grado di svolgere certi servizi (operazioni)» [fonte: 02, sl. 4]. Per rendere visibile il cambio di prospettiva si riusa «la notazione puntata già in uso per le struct, attribuendole però nuovo significato» [fonte: 02, sl. 5]: il punto non seleziona più solo un campo, ma anche «un'operazione fra quelle offerte da un'entità». Richiedere un servizio in questo modo si chiama invocare un **metodo** [fonte: 02, sl. 6].

L'esempio da tenere a mente è quello che scriverai mille volte:

```
System.out.println("Hello!");
```

«richiede al componente System.out di svolgere il servizio println», e out «è a sua volta un componente dell'entità System» [fonte: 02, sl. 6]. Il primo punto scende dentro System fino al componente out; il secondo sceglie il servizio. Letta così, la riga ha la stessa forma comp.operation(argomenti), con comp = System.out.

La visione. È il seguito diretto del modulo 01: lì il problema era passare dall'*in-the-small* all'*in-the-large*; qui si vede il suo effetto sul linguaggio. Quando le entità sono molte, il destinatario non è più ovvio e la sintassi lo mette in testa. Il docente nota che questo modo di pensare non è estraneo: col doppio clic su un'icona «vi concentrate sul "chi" deve fare qualcosa, non sull'operazione», e non aprite il programma per poi scegliere l'operazione da menù [fonte: 02, sl. 4].

△ **Distinzione da non collassare** (errori.md, pattern 1). *Notazione puntata sulle struct del C e notazione puntata in Java* hanno la stessa forma ma non lo stesso significato: nella prima il punto seleziona un campo, nella seconda può selezionare anche un servizio. Caso limite che le separa: System.out.println(...), dove dopo l'ultimo punto c'è un'operazione con argomenti, cosa che in una struct non può esistere.

2. Com'è fatta un'applicazione a oggetti: entità statiche e dinamiche

Cosa produce. Una mappa di cosa c'è dentro un programma Java, e quindi la prima risposta operativa alla domanda «da cosa è fatto il linguaggio» rimasta aperta dopo 01.

Cosa c'è dietro. «Una applicazione è strutturata come un insieme di entità» [fonte: 02, sl. 7-9], di due specie:

- alcune sono **statiche**, «ossia esistono prima dell'inizio del programma e permangono per tutta la sua durata»: librerie «prive di stato, es. libreria matematica», moduli software statici («oggetti singoli»), definizioni di tipi. Nella sl. 9 il docente dà loro il nome: **classi o oggetti singleton**;
- altre sono **dinamiche**, «ossia vengono create durante l'esecuzione solo al momento del bisogno»: sono gli **oggetti**.

E poiché «ogni applicazione deve avere un punto di partenza prestabilito, una di tali entità statiche contiene il main» [fonte: 02, sl. 7].

La visione. In questo modulo compaiono solo le entità statiche: ogni esempio (MyProg, Esempio1, Esempio2) è una classe che contiene un main e usa altre entità statiche già pronte nell'infrastruttura, System e Math. Gli oggetti dinamici — quelli creati al momento del bisogno — non compaiono ancora: arrivano con 04b (*Classi e oggetti*). Tieni presente questa asimmetria mentre leggi il main: il main sta in un'entità che esiste *prima* che il programma parta, e la parola static nella sua firma va letta tenendo presente questo — il modulo non la spiega esplicitamente (sezione 4).

3. Perché Java è fatto così: i requisiti dei nuovi linguaggi

Cosa produce. Il criterio con cui leggere tutte le differenze dal C che seguono.

Cosa c'è dietro. Java, C#, Scala e Kotlin «sono linguaggi progettati ex novo, facendo tesoro delle esperienze (e degli errori) precedenti», ispirati a C e C++ «ma senza il requisito della piena compatibilità all'indietro» [fonte: 02, sl. 10]. Due obiettivi:

- **Obiettivo 1:** sostituire costrutti «poco chiari, sintatticamente obsoleti ed error-prone» con costrutti «intrinsecamente più sicuri, chiari, di più alto livello», che evitino «la gestione diretta (error-prone) di tanti/troppi dettagli»;
- **Obiettivo 2:** «intercettare a compile-time quanti più errori possibile: “se si compila, molto probabilmente è ok”» [fonte: 02, sl. 10].

Le idee concrete [fonte: 02, sl. 11]: sostituzione dei puntatori con **riferimenti**, dereferenziazione automatica, allocazione e deallocazione automatica della memoria, e **type safety** definita come «type system stringente + type inference + controlli a run-time». Più recentemente, soprattutto in Scala e Kotlin: null safety, type inference più evoluta, distinzione valori/variabili, preferenza per strutture immutabili, funzioni come *first-class entities* e lambda expression, stile più funzionale.

La visione. L'obiettivo 2 è la ragione per cui gli errori di tipo che vedrai nella sezione 7 sono **errori di compilazione**, non di esecuzione. La definizione di type safety, però, dice tre cose, e la terza è «controlli a run-time»: il compilatore non intercetta tutto. «Molto probabilmente» non è «sicuramente».

△ **Compilazione ed esecuzione restano due fasi** (errori.md, FI2, 2026-09-15, candidato trasversale). «Se si compila, molto probabilmente è ok» non vuol dire che ciò che compila non possa più sbagliare: resta lo spazio dei controlli a run-time, che è esattamente dove nasce l'*error* di JUnit — il punto rimasto debole nella verifica di 01. Il compilatore sposta errori dalla seconda fase alla prima; non abolisce la seconda.

4. Il main: quattro differenze dal C

Cosa produce. La forma canonica della più semplice applicazione Java:

```
public class MyProg {
    public static void main(String[] args){
        int x=3, y=4; int z = x+y;
    }
}
```

[fonte: 02, sl. 15, 17]

Cosa c'è dietro. «La più semplice applicazione possibile è costituita da un singolo componente (*singleton*), che definisce soltanto il main» [fonte: 02, sl. 12]. Il docente elenca le differenze rispetto al C una per una:

1. **Il main sta dentro una classe.** In C «è semplicemente scritto in un file, non è racchiuso in alcun costrutto linguistico»; qui «dev'essere posto in una *classe pubblica* ed essere esso stesso *pubblico* (criteri di protezione)» [fonte: 02, sl. 12]. È la sezione 2 resa sintassi: il punto di partenza sta dentro un'entità statica.
2. **Un solo argomento.** In C il main «può avere o non avere argomenti, argc / argv»; nei nuovi linguaggi «ha sempre come unico argomento un singolo oggetto: un array di stringhe» [fonte: 02, sl. 14]. Nella sl. 15 il docente evidenzia la «nuova posizione per le parentesi quadre []»: `String[] args`, il tipo è «array di String». (Nella sl. 41 lo stesso docente scrive `String args[]`, con le parentesi alla maniera del C.)
3. **Tipo di ritorno.** In C `void` o `int`; «in Java, il main ha sempre tipo di ritorno `void` (NON `int`)»; in C# può essere `void` o `int`; in Scala e Kotlin è `Unit` [fonte: 02, sl. 14].
4. **Definizione delle funzioni.** Solo Scala e Kotlin le definiscono diversamente dal C, con `def` / `fun` e il tipo postfisso: `def main(args: Array[String]):Unit = {...}` [fonte: 02, sl. 14, 16]. Java su questo resta vicino al C.

Anche le variabili seguono il C: «Java e C# ammettono l'assegnamento di più variabili in un'unica istruzione» (`int x=3, y=4;`), mentre in Scala e Kotlin le variabili si introducono con `var` (o `val` se imm modificabili), il tipo è postfisso (`var x:Int = 3`) e ogni variabile va definita separatamente [fonte: 02, sl. 17-18].

La visione. Delle quattro parole prima di `main`, questo modulo spiega `public` («criteri di protezione») e `void` (Java: solo `void`). Il perché di `static` non è spiegato esplicitamente nelle slide di questo modulo: va collegato alla distinzione statico/dinamico della sezione 2. Lo riprende il riassunto dell'esercitazione 02x, e il modulo che lo sviluppa è 04b. Le quattro differenze non sono dettagli di sintassi slegati: la prima e la seconda vengono dall'architettura a entità (il punto di partenza sta dentro una classe, e riceve un oggetto), la terza e la quarta da quanto ciascun linguaggio si è allontanato dal C.

5. Convenzioni di naming: il nome del file non è libero

Cosa produce. La regola che decide come si chiama ogni file che crei.

Cosa c'è dietro. Le convenzioni generali [fonte: 02, sl. 19]:

- una **classe** ha un nome «chiaro ed espressivo, che inizi per maiuscola e segua la convenzione CamelCase»;

- **funzioni e variabili** hanno un nome chiaro ed espressivo che inizia per minuscola (tranne C#) e segue poi CamelCase;
- le **costanti** hanno un nome tutto maiuscolo;
- «Non è gradito l'uso dell'underscore (_)».

Sulla corrispondenza file/classe i linguaggi divergono, e qui va tenuta la distinzione fra *raccomandazione* e *regola*. In generale «un file dovrebbe contenere una sola classe e il nome del file dovrebbe coincidere con nome della classe»; ma **in Java questa regola è rigida**: «ogni file può contenere una sola classe pubblica (può poi contenerne altre non pubbliche)» e «il file deve chiamarsi esattamente come la classe e avere estensione .java» [fonte: 02, sl. 20-21]. In C#, Scala e Kotlin è solo raccomandato, e più classi pubbliche possono stare nello stesso file. «NB: «esattamente come la classe» significa maiuscole/minuscole comprese, senza eccezioni» [fonte: 02, sl. 21].

La visione. In Java il nome del file è parte del programma, non un'etichetta: MyProg in myprog.java non è una questione di stile. È lo stesso nome che userai per eseguire la classe (sezione 6) e che Eclipse ti impone quando crei una classe.

△ **Dovrebbe / deve** (errori.md, pattern 1 e pattern 4). Le slide usano «dovrebbe» per la raccomandazione generale e «deve» per Java: riportale con il verbo giusto. Caso limite che le separa: due classi public nello stesso file sono ammesse in Kotlin, vietate in Java.

6. Compilazione ed esecuzione sull'infrastruttura: javac e java

Cosa produce. Un programma che parte dalla riga di comando.

```
C:> javac Esempio1.java           produce Esempio1.class
C:> java Esempio1 alfa beta gamma
alfa
```

[fonte: 02, sl. 29, 31], con il programma:

```
public class Esempio1 {
    public static void main(String[] args){
        System.out.println(args[0]);
    }
}
```

[fonte: 02, sl. 27]

Cosa c'è dietro. Il confronto con C e C# è costruito per far vedere una differenza che la forma dei comandi nasconde [fonte: 02, sl. 22]:

- cc MyProg.c produce MyProg.exe: «L'EXE ottenuto è eseguibile direttamente sullo specifico sistema operativo»;
- javac MyProg.java produce MyProg.class: «Il file ottenuto è eseguibile sull'infrastruttura Java» — e il docente annota a fianco: «**Non sono la stessa cosa!**». Prerequisito: «dev'essere installato il JDK e dev'essere nel PATH»;
- csc MyProg.cs produce ancora un .exe, ma «eseguibile sull'infrastruttura .NET».

Da qui l'esecuzione [fonte: 02, sl. 30]: in C si esegue «l'eseguibile autocontenuto prodotto dal compilatore»; negli altri linguaggi si **invoca l'infrastruttura specificando la classe che contiene il main**. In Java l'infrastruttura si invoca con l'«interprete (strato-ponte) java». Nota cosa riceve java: Esempio1, il nome della **classe**, non il file Esempio1.class.

Due dettagli che nel C erano diversi:

- **args[0] non è il nome del programma**: «In Java e C#, args[0] non è il nome del programma: è già il primo argomento» [fonte: 02, sl. 27]. Con java Esempio1 alfa beta gamma stampa alfa. Nel C lo stesso ruolo lo avrebbe argv[1], ed è infatti quello che la versione C della slide stampa;
- **il caso C#** mostra perché «eseguibile» è relativo: Esempio1 alfa beta gamma «sembra uguale al primo, ma non funziona se sulla macchina non è installato il .NET Framework» [fonte: 02, sl. 31, nota 1].

Scala e Kotlin sulla JVM. Sono «costruiti per funzionare sulla stessa infrastruttura di Java: la Java Virtual Machine (JVM)». Si compilano con i loro compilatori (scalac, kotlinc), «ma il risultato è comunque costituito da file .class», perché «Kotlin e Scala sono basati sulla piattaforma Java (con cui tra l'altro sono interoperabili)»; possono però compilare anche per altre piattaforme [fonte: 02, sl. 23]. I nomi prodotti non sempre coincidono: scalac produce Esempio1.class **ed** Esempio1\$.class, kotlinc produce Esempio1Kt.class, e quindi si esegue con kotlin Esempio1Kt [fonte: 02, sl. 29, 31]. L'esecuzione avviene sull'infrastruttura Java «+ librerie Scala» o «+ librerie Kotlin» [fonte: 02, sl. 24, 35].

La visione. È la definizione di 01 messa in pratica: «eseguibile» è sempre relativo allo strato che interpreta il file. Il .class non è un EXE di serie B: è un eseguibile per un'altra macchina, la JVM, e java è il ponte fra le due. Scala e Kotlin lo dimostrano dall'altro lato: linguaggi diversi, stesso formato di uscita, stessa infrastruttura. All'esame lavorerai in Eclipse (fonti.md), che compila ed esegue al posto tuo; ma quando un progetto «non parte» la domanda è sempre la stessa: il problema è nella compilazione (niente .class) o nell'esecuzione (il .class c'è, qualcosa va storto quando la JVM lo esegue)?

△ **Compilare ≠ eseguire, javac ≠ java** (errori.md, FI2). Sono due programmi diversi, con due input diversi (un file .java; un nome di classe) e due momenti diversi. Il primo richiede il JDK; il secondo solo l'infrastruttura. È la stessa separazione che in 01 regge la coppia JDK/JRE e la coppia *failure/error*.

7. Spazi di nomi: Math.sin contro sin

Cosa produce. Il terzo esempio del docente, con tre entità: la classe col main, il componente di stampa e la «libreria matematica» [fonte: 02, sl. 32].

```
public class Esempio2 {  
    public static void main(String[] args){  
        System.out.println( Math.sin(Math.PI/3) );  
    }  
}
```

che stampa 0,866025403784439 [fonte: 02, sl. 33, 35].

Cosa c'è dietro. In Java la libreria matematica è «l'entità (classe): Math», e offre costanti (e, π) e «decine di funzioni utili». In C#, Scala e Kotlin Math (o math) «è un sotto-componente di qualcos'altro» (System.Math, scala.math, kotlin.math); «in Java no» [fonte: 02, sl. 32]. Il confronto con il C è annotato direttamente sul codice [fonte: 02, sl. 33]:

- in C, `#include <math.h>` e poi `sin(M_PI/3)`: «**Spazio di nomi unico = rischio di clash + necessità di differenziare tutti i nomi**»;
- in Java, `Math.sin(Math.PI/3)`: «**Spazio di nomi intrinsecamente modulare = no clash**».

La visione. È la sezione 1 applicata ai nomi. Se ogni servizio si chiede *a qualcuno*, il nome del servizio vive dentro il suo destinatario: `Math.sin` e un eventuale `sin` di un'altra classe non si toccano. In C, dove tutto sta in un unico spazio, due librerie che definiscono la stessa funzione collidono. È anche il motivo per cui Scala e Kotlin possono usare direttamente `Math.sin(Math.PI/3)` di Java: sono interoperabili e il nome qualificato indica senza ambiguità «il componente Java dell'infrastruttura sottostante» [fonte: 02, sl. 34]. Il tema dei nomi modulari si sviluppa in 08 (*Package e namespace*).

8. La documentazione generata: javadoc

Cosa produce. Un manuale HTML ricavato dai commenti del sorgente:

```
javadoc -d docs Esempio0.java
```

«Produce nella cartella docs un manuale HTML» [fonte: 02, sl. 42].

Cosa c'è dietro. Il ragionamento del docente parte da un fatto, non da un principio: un buon programma dovrebbe essere ben documentato, «...ma l'esperienza insegna che quasi mai ciò viene fatto!» («non c'è tempo», «ci si penserà poi»). Allora «Java prende atto che la gente non scrive documentazione e quindi fornisce uno strumento per produrla automaticamente a partire da particolari commenti nel programma: javadoc» [fonte: 02, sl. 39]. Un commento Javadoc «inizia con `/**` (anziché `/*`)», termina normalmente con `*/`, e può stare in testa a una classe o a singole funzioni. Dentro, tag come `@author` e `@version` portano «informazioni di documentazione che verranno estratte», ma «vengono inseriti solo a richiesta» [fonte: 02, sl. 41-42].

Gli analoghi negli altri linguaggi: Scaladoc («segue al 99% la stessa sintassi di Javadoc»), Kdoc (mix fra Javadoc e markdown); in C# il compilatore estrae da commenti `///` un file XML, che altri strumenti trasformano in manuali [fonte: 02, sl. 40].

La visione. È la quarta voce del «prodotto industriale» di 01 — l'automazione dei passi chiave — applicata alla documentazione: se una cosa non la fa nessuno, la si attacca al codice e la si fa produrre a uno strumento. Il ragionamento è quello che il docente userà anche per il collaudo: non contare sulla buona volontà, ma rendere il passo parte del processo.

9. Tipi base: primitivi sì o no, e il boolean

Cosa produce. L'elenco dei tipi con cui si costruisce ogni esercizio, e due regole che in C non esistevano.

Cosa c'è dietro — primitivi. «Java mantiene la nozione di tipo primitivo del C, pur estendendoli e ridefinendoli»: un approccio «conservativo» dovuto a ragioni storiche (chi veniva dal C era abituato) e di prestazioni, all'epoca — «MA l'esperienza ha dimostrato che non è stata una grande idea!» [fonte: 02, sl. 44]. I linguaggi successivi li sostituiscono con tipi di oggetti, per «uniformità & drastica semplificazione»: in Scala e Kotlin si vede dal nome (`int` → `Int`), e C# li presenta come «tipi primitivi mascherati».

Cosa c'è dietro — boolean. «Un boolean non è più sinonimo di «intero 0/1»»: è «un tipo autonomo, totalmente disaccoppiato dagli interi». Le espressioni relazionali e logiche danno un boolean, «non un `int` come in C», e «intenzionalmente non si convertono boolean in interi e viceversa, neanche con cast (bisogna scriversi funzioni apposite)». Gli unici valori ammessi sono `false` e `true`, «che non sono 0 e 1» [fonte: 02, sl. 45]. In Java il tipo è `boolean` (primitivo). Solo C# offre, nel componente `Convert`, funzioni come `Convert.ToInt16(bool)` e `Convert.ToBoolean(int)` che coprono questa conversione [fonte: 02, sl. 46].

Interi e reali in Java [fonte: 02, sl. 47-48]:

Tipo	Byte	Note dalla slide
<code>byte</code>	1	-128 ... +127
<code>short</code>	2	-32768 ... +32767
<code>int</code>	4	circa $\pm 2 \cdot 10^9$
<code>long</code>	8	circa $\pm 9 \cdot 10^{18}$ · le costanti terminano con L
<code>float</code>	4	IEEE-754 · circa 6-7 cifre significative · le costanti terminano con F
<code>double</code>	8	IEEE-754 · circa 14-15 cifre significative

Solo C# ha interi senza segno (`byte`, `ushort`, `uint`, `ulong`) e il tipo `decimal`, fuori standard IEEE-754, a 16 byte in base 10: molto preciso e adatto ai calcoli finanziari, ma circa 20 volte più lento e con range più ridotto [fonte: 02, sl. 47-48].

La visione. Il `boolean` è l'obiettivo 1 della sezione 3 in un caso concreto: in C «vero» e «falso» sono interi travestiti, e ogni intero può fare da condizione; in Java il tipo separato impedisce di mescolare i due mondi senza dichiararlo scrivendo una funzione. La questione «primitivi sì o no» tornerà: il docente la chiama «non una grande idea», e il costo si vedrà quando servirà trattare un `int` come un oggetto — modulo 16, *Wrapper per tipi primitivi*.

△ **In Java `char`, `int` e `boolean` non stanno sullo stesso piano** (`errori.md`, pattern 1). Fra `char` e intero Java converte automaticamente (sezione 11); fra `boolean` e intero no, «neanche con cast». Caso limite: `(int) true` non compila.

10. Compatibilità fra reali: implicito, esplicito, Design Intent

Cosa produce. La regola che decide se un assegnamento fra tipi reali compila.

```
double x = 3.54F;           // OK
double z = Math.sin(Math.PI/3); // OK
float f = 3.54;             // NO! errore di compilazione
float f = Math.sin(Math.PI/3); // NO!
float f = (float) 3.54;      // OK: cast esplicito
```

[fonte: 02, sl. 50, 55]. L'errore del compilatore è: *Possible loss of precision — Found double, required float* [fonte: 02, sl. 50].

Cosa c'è dietro. «In Java, C#, Scala sono ammessi solo gli assegnamenti che non causano perdita di informazione» [fonte: 02, sl. 49]:

- `double x = 3.54F;` è lecita: «da float a double non si perde precisione»;
- `float f = 3.54;` è illecita: «da double a float si perderebbe precisione».

Nota cosa rende illecita la seconda riga: il letterale 3.54 senza suffisso è un `double` (da qui «Found double»); con la F diventa `float` (sl. 48). E `Math.sin` restituisce un `double`, per cui la seconda riga errata è lo stesso caso.

Quando la perdita è **voluta**, bisogna dirlo: «Se si vuole consapevolmente usare un `float` per memorizzare un valore `double`, accettando la perdita di precisione che ne deriverà, occorre asserirlo esplicitamente» — in Java e C# «con un cast», che si scrive mettendo il tipo target fra parentesi davanti all'espressione, `(float) 3.54` [fonte: 02, sl. 54–55]. Il motivo: «per consentire un'operazione potenzialmente rischiosa, occorre che il progettista renda esplicito il suo **Design Intent**», ossia «dica chiaramente, scrivendo qualcosa, che ciò non è il frutto di una svista, ma è suo preciso intendimento» [fonte: 02, sl. 54].

Kotlin porta la regola all'estremo. «In Kotlin invece le conversioni implicite non sono mai ammesse: è una scelta di progetto!» [fonte: 02, sl. 49]. Anche l'esempio corretto diventa errato: `val z:Double = 3.54F;` non compila, e serve `3.54F.toDouble()`. La ragione: «prevale l'idea che le conversioni debbano essere esplicite anche quando non c'è perdita di informazione, per far emergere sempre il Design Intent del progettista» — «un approccio moderno, ispirato dal principio di type safety» [fonte: 02, sl. 53]. In Scala e Kotlin il cast non esiste: si usano funzioni `toXXX()`, che in Scala si scrivono senza parentesi finali «in ossequio al principio di accesso uniforme» (`3.54.toFloat()`), in Kotlin con `(3.54.toFloat())` [fonte: 02, sl. 55–56].

La visione. Tre posizioni sullo stesso asse, e Java sta nel mezzo: il C converte in silenzio in entrambe le direzioni; Java converte in silenzio solo quando è sicuro e vuole un cast quando non lo è; Kotlin vuole sempre la conversione scritta. Il principio che le ordina è quello della sezione 3: portare a compile-time la decisione, e far sì che un errore possibile venga o segnalato dal compilatore o firmato dal progettista. Un cast in Java non è un modo per «far tacere» il compilatore: è una dichiarazione che la perdita è voluta.

△ **Due distinzioni da tenere separate** (errori.md, pattern 1). - *Implicita / esplicita* non coincide con *lecita / illecita*: una conversione implicita è lecita in Java (`double x = 3.54F`) ma illecita in Kotlin; una esplicita è sempre lecita, ma non per questo innocua. - *Il verso conta*: `float` → `double` passa, `double` → `float` no. Il criterio non è «tipi reali compatibili» ma «si perde informazione?».

△ **Far quadrare prima di concludere** (errori.md, pattern 2). Davanti a `float f = 3.54`; la prima impressione è «3.54 è un reale, f è un reale, va bene». Il dato che la smentisce è nel suffisso che manca.

11. Caratteri: da Unicode a UTF

Cosa produce. La risposta a «cos'è un char in Java», e a perché un testo con le lettere accentate può apparire illeggibile su un altro computer.

Cosa c'è dietro — il char. «A differenza del C, «carattere» non è più sinonimo di «byte»»: 127 caratteri «non bastano più da un sacco di tempo», e «il mondo non ospita solo le culture occidentali». Il nuovo approccio: «un «carattere» di 2 byte (UTF-16)», con i primi 127 caratteri uguali ad ASCII [fonte: 02, sl. 60]. In Java il tipo è `char`, primitivo; in Scala e Kotlin `Char`, tipo di oggetto. Le conversioni carattere ↔ intero: in Java e Scala **automatiche**, in C# implicite tramite cast nel verso `int` → `char`, in Kotlin **esplicite** con `toXXX()` [fonte: 02, sl. 65]. Nell'esempio Java della slide, `char ch = 'A'; int x = ch;` compila senza cast.

Cosa c'è dietro — Unicode. Lo standard Unicode copre l'intervallo da 000000H a 10FFFFH, 1.114.112 caratteri, «suddivisi in 17 «piani» da 65.536 caratteri ciascuno» [fonte: 02, sl. 61]:

- **piano 0, Basic Multilingual** (0000–FFFF): quello usato quasi sempre;
- **piani 1 e successivi, Supplementary**: ideogrammi rari, caratteri storici, e le emoji, con code point «da 1F600H in poi» [fonte: 02, sl. 64].

Un carattere, o **code point**, si indica con `U+nnnn`; i caratteri che non sono sulle tastiere si specificano con una «codifica semi-numerica», `'\u2122'` [fonte: 02, sl. 61, 66]. L'intervallo D800–DFFF del piano 0 «non è assegnato (serve per UTF-16)».

Cosa c'è dietro — UTF. Qui sta la distinzione che regge tutto il resto: «Unicode però si limita ad assegnare codici ai caratteri: **non dice come debbano essere mappati su sequenze di byte**». Questo lo fa **UTF**, *Unicode Transformation Format*: «a mapping from every Unicode code point to a unique byte sequence» [fonte: 02, sl. 66]. Tre codifiche:

- **UTF-8**, lunghezza variabile da 1 a 4 byte: 1 byte per i primi 128 caratteri («compatibile ASCII»), 2 per i successivi 1920, 3 per il resto del Basic Multilingual, 4 per gli altri piani, «tra cui molte Emoji». Molto usato per testo ed email [fonte: 02, sl. 66, 68];
- **UTF-16**, 2 byte per il Basic Multilingual e 4 per gli altri piani. I caratteri a 4 byte si esprimono come **coppia surrogata**, due valori nel range riservato D800–DFFF, che è ciò che permette di distinguerli da quelli a 2 byte. «Più complesso ma

efficiente → usato in Java, .NET, macOS» [fonte: 02, sl. 70–71]. L'ordine dei byte in memoria può variare (big / little endian), e \uFEFF è il marcatore che permette di riconoscerlo [fonte: 02, sl. 71];

- **UTF-32**, sempre 4 byte: «molto semplice, MA usa una quantità sproporzionata di memoria», con un vantaggio «più apparente che reale» [fonte: 02, sl. 72].

La tabella del docente, sullo stesso carattere in tre codifiche [fonte: 02, sl. 68, 70]:

Carattere	Code point	UTF-8	UTF-16
\$	U+0024	1 byte: 00100100	2 byte
£	U+00A3	2 byte: 11000010 10100011	2 byte
€	U+20AC	3 byte	2 byte
emoji 😊	U+1F608	4 byte	4 byte: coppia surrogata \uD83D \uDE08

La visione. Unicode è la risposta a «quali caratteri esistono», UTF a «come li scrivo su disco o in memoria». Storicamente «ogni piattaforma faceva un po' da sé» — ASCII per tutti ma solo per 127 caratteri inglesi, poi standard incompatibili, persino sul ritorno a capo (CR o CR+LF) — e UTF è «una sorta di «lingua franca» per far interoperare macchine e piattaforme anche molto diverse» [fonte: 02, sl. 67]. La conseguenza pratica è la più banale: «se un testo non è UTF e lo condividi con qualcun altro (che magari ha un Mac mentre tu hai Windows), molti caratteri risulteranno «sbagliati» o illeggibili – a partire dalle lettere accentate!» [fonte: 02, sl. 67]. Le sl. 73–74 mostrano un editor con un sorgente Java salvato in codifica Windows 1252 e poi convertito in UTF-8 dal menù di codifica. Il tema riappare quando si leggono e scrivono file di testo, in 27–29 (*Gestione I/O*).

⚠ **Code point ≠ byte, Unicode ≠ UTF** (errori.md, pattern 1). È la distinzione su cui il modulo è costruito, ed è del tipo che tendi a fondere: «Unicode a 16 bit» è sbagliato due volte. Unicode non ha un numero di bit, perché assegna solo numeri; e i numeri arrivano a 21 bit (10FFFFH). Caso limite che separa le coppie: l'emoji U+1F608 è **un** code point, ma in UTF-16 occupa **due** unità da 2 byte (\uD83D \uDE08). Da qui la conseguenza che non va data per scontata: un char Java è un'unità UTF-16 da 2 byte, e per un carattere supplementare **un char solo non basta**. Il docente lo dice per i messaggi: le emoji «sono supplementary character, occupano l'equivalente di 4 caratteri standard ciascuno» [fonte: 02, sl. 69].

Casi limite

- **Il nome da dare a java.** All'interprete si passa la classe che contiene il main (java Esempio1), non il file Esempio1.class (sl. 30–31). E il .class generato da kotlinc per Esempio1.kt si chiama Esempio1Kt: il nome da passare all'interprete non è sempre il nome del sorgente (sl. 29, 31).

- **La classe public nel file col nome sbagliato.** In Java non è uno stile scorretto, è un vincolo: il file deve chiamarsi esattamente come la classe pubblica, maiuscole comprese (sl. 20-21).
 - **args[0] con nessun argomento.** L'esempio 1 presuppone almeno un argomento: args[0] è il primo argomento, non il nome del programma (sl. 27). Lanciato senza argomenti, il codice compila e il problema emerge solo in esecuzione — il «molto probabilmente» della sl. 10.
 - **Il letterale reale senza suffisso.** 3.54 è un double; float f = 3.54; non compila (sl. 50). Serve 3.54F o il cast (float).
 - **Il cast che non esiste.** Fra boolean e interi non c'è conversione, «neanche con cast» (sl. 45): la via del Design Intent qui non è disponibile.
 - **L'emoji in un char.** Un carattere supplementare non sta in un singolo char UTF-16: serve la coppia surrogata (sl. 70).
-

Connessioni

- **Con 01 (chiuso):** la sl. 22 («Non sono la stessa cosa!», EXE contro .class) è la definizione di «eseguibile relativo all'infrastruttura» di 01 vista sul singolo file, prima del JAR. La nota «dev'essere installato il JDK» per javac ripete la distinzione JDK/JRE: per compilare serve il kit, per eseguire basta l'infrastruttura. E la sezione 3 («se si compila, molto probabilmente è ok», «controlli a run-time») è la radice della coppia *failure/error*, il punto da verificare per primo al ripasso di 01 del 2026-09-19.
 - **Con 02x (Esercitazione: tipi base) e 02z (Addendum: main in Java 21):** 02x è la pratica di questo modulo (percorso.md, *Mappa teoria → pratica*) e il suo riassunto dice perché il main è statico: «deve esistere dall'inizio alla fine del programma». 02z aggiorna la forma del main della sezione 4. Dopo 03 si apre anche LAB01 (linea di comando).
 - **Con 04b (Classi e oggetti):** le entità «dinamiche» della sl. 8, qui solo nominate, e il significato di static nella firma del main. È anche il modulo che, con 02 e 12, costruisce la «struttura formale» di Java rimasta aperta dopo 01 (stato/corrente.md, punto 1b).
 - **Con 08 (Package e namespace):** lo «spazio di nomi intrinsecamente modulare» di Math.sin (sl. 33) è introdotto qui come contrasto col C; il meccanismo è materia di 08.
 - **Con 16 (Wrapper per tipi primitivi):** il giudizio del docente sui primitivi, «non è stata una grande idea» (sl. 44), si paga lì. Aggancio per titolo, da verificare all'apertura.
 - **Con CALC — Calcolatori Elettronici T (S1):** la rappresentazione in byte di interi e reali (tabella sl. 47-48, IEEE-754) e l'ordine dei byte big/little endian che la sl. 71 incontra su UTF-16 sono materia di rappresentazione dell'informazione. Da precisare con le fonti di CALC quando si apre il corso, non da dare per acquisito qui.
-

Domande di autoverifica

Da rispondere senza riaprire la lezione né le slide.

1. Perché i linguaggi a oggetti scrivono `comp.operation(argomenti)` invece di `operation(comp, argomenti)`? In che tipo di sistema la notazione del C è «naturale», e dove mostra i suoi limiti?
 2. Hai `Esempio1.java` con il `main` che stampa `args[0]`. Quali due comandi usi per ottenere alfa a video, cosa produce il primo, cosa riceve il secondo, e cosa serve installato per ciascuno?
 3. Il docente scrive che `cc` e `javac` producono due file che «non sono la stessa cosa». Cosa li distingue? E perché l'EXE prodotto da `csc` conferma la stessa idea?
 4. `float f = 3.54;` non compila, `double x = 3.54F;` sì. Spiega perché, come rendi lecita la prima e cosa significa farlo. Cosa cambia in Kotlin, e perché?
 5. Qual è la differenza fra Unicode e UTF? Quanti `char` Java servono per rappresentare il code point `U+1F608`, e perché?
-

Riepilogo

Cosa cambia, per il programmatore, passando dal punto di vista procedurale a quello a oggetti? L'enfasi passa dall'operazione all'entità che la svolge. In un sistema con molte entità il problema è distribuire le responsabilità, e quindi diventa essenziale dire a *chi* ci si rivolge: `System.out.println(...)` chiede al componente `System.out` il servizio `println`. Un'applicazione è un insieme di entità statiche (classi, singleton, una delle quali contiene il `main`) e dinamiche (oggetti creati al bisogno).

Che cosa succede fra il sorgente e l'esecuzione di un programma Java? `javac` (dal JDK) compila `MyProg.java` in `MyProg.class`, che non è un eseguibile per il sistema operativo ma per l'infrastruttura Java; `java MyProg` invoca l'infrastruttura passandole la classe che contiene il `main`. Il file deve chiamarsi esattamente come la classe pubblica, e `args[0]` è già il primo argomento. Scala e Kotlin producono gli stessi `.class` e girano sulla stessa JVM.

Perché Java rifiuta `float f = 3.54;` ma accetta un `cast`? Perché i nuovi linguaggi vogliono intercettare a compile-time quanti più errori possibile: le conversioni implicite sono ammesse solo senza perdita di informazione, e quando la perdita è voluta il progettista deve dichiarare il suo *Design Intent* con un `cast`. Kotlin va oltre e vuole la conversione esplicita sempre. Lo stesso spirito separa `boolean` dagli interi e fa del `char` un'unità UTF-16 da 2 byte: Unicode numera i caratteri, UTF decide come diventano byte.